

DataPipe

User Manual

Visual data flows for industrial data — from machine to database in minutes

Version 0.10 (development state after Increment 10) · July 2026

All screenshots in this manual were captured from a running DataPipe installation.

1. What is DataPipe?

DataPipe is a web-based visual data flow platform. You build **flows** by dragging **nodes** onto a canvas, wiring them together, and deploying them to a **runtime** that executes them. Data travels between nodes as **datagrams**: self-describing messages that carry the payload plus metadata such as source, timestamps, and a quality indicator.

A flow runs in one of two modes, determined automatically by its entry nodes:

- **Streaming flows** start with a plain source (MQTT In, Schedule, SQL Source, ...) and run continuously after deployment — the right mode for machine data acquisition.
- **Triggered workflows** start with a trigger node (HTTP In webhook, Error Trigger) and create one tracked, inspectable **execution** per event — with history, per-node input/output capture, and re-run.

Between sources and sinks sits a processor library: a sandboxed JavaScript script node, calculator, window/aggregate, switch, filter, merge/join, split/batch, loop, delay/throttle, lookup, state, and try/catch — all sharing one expression language. Connectors cover MQTT, HTTP/REST, WebSocket, files (CSV/TSV, JSON, XML, Excel), SQL databases (PostgreSQL, MySQL, MSSQL, SQLite), MongoDB, Redis, Kafka, S3-compatible object storage, raw TCP/UDP/Serial, Modbus, and OPC-UA.

Roles. What you can do depends on your role per project: **Viewer** (read only, no payload inspection), **Operator** (operate flows and inspect live data), **Editor** (create, edit, deploy), **Project Admin** (manage the project), **System Admin** (everything).

2. Signing in

Open the editor in your browser and sign in with the username and password from your administrator. The top bar always offers: language switch (English/Deutsch), light/dark theme toggle, and sign out. The **Fleet** and **Monitoring** links in the top bar lead to the operations pages described in section 8.

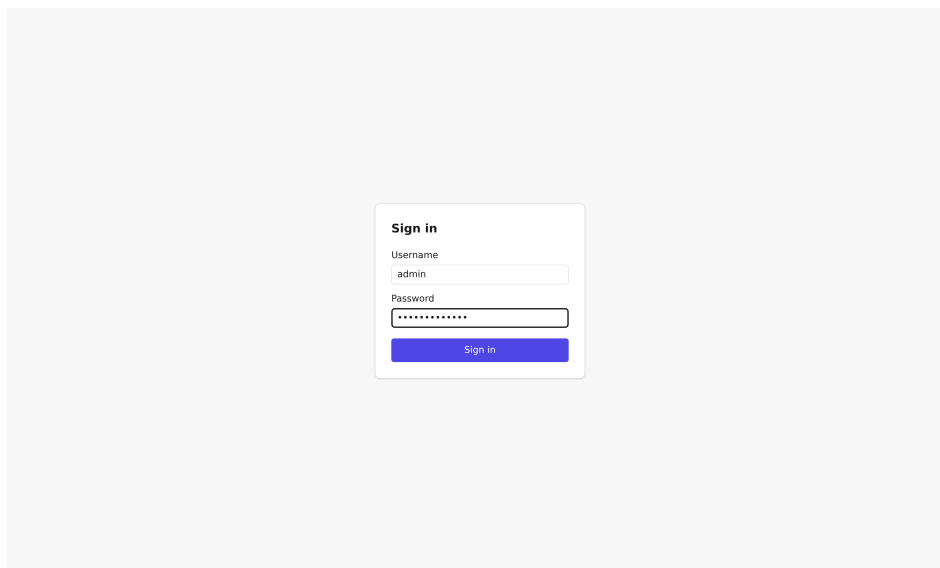


Figure 1 — The sign-in page.

3. Projects, flows, and connections

After signing in you land on the **Projects** page. A project groups flows, connections, credentials, and environment profiles for one team, plant, or purpose — and is the unit access control applies to.

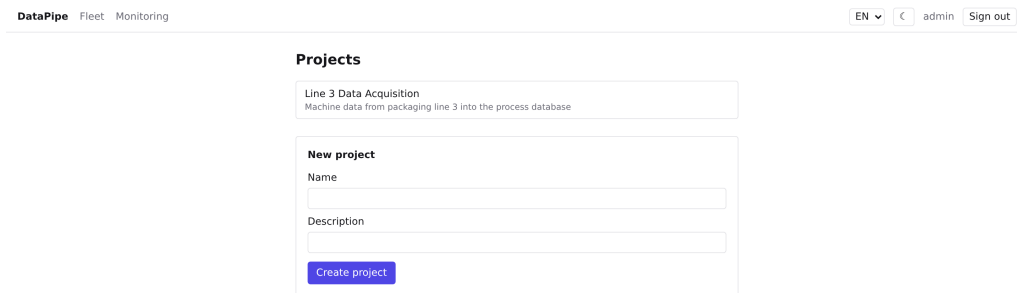


Figure 2 — The Projects page: open an existing project or create a new one.

Opening a project shows its **flows** (with deployment state), its **connections**, its **environment profiles**, and a **template gallery** of importable starter flows. **Export project / Import** moves flows between installations as portable JSON — always secret-free: exports reference connections by name and type but never contain credential values.

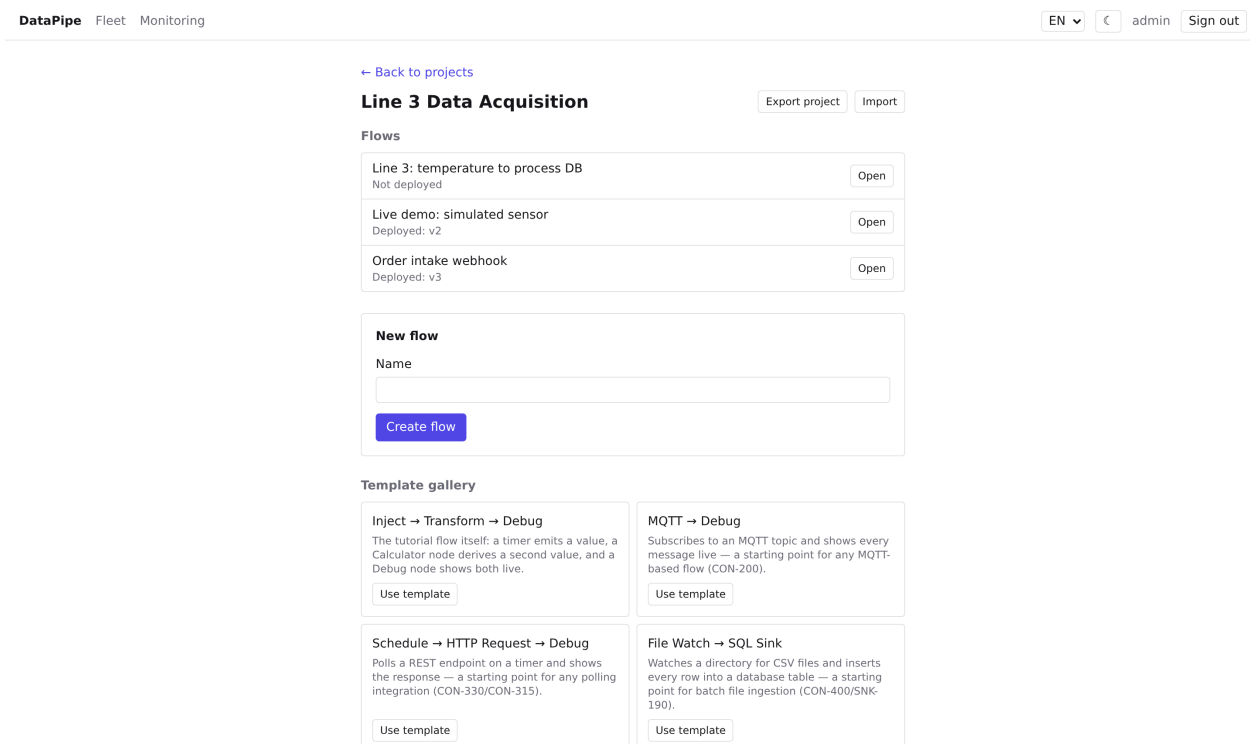


Figure 3 — A project page: flows with deployment state, new-flow form, template gallery.

Connections. A connection is a named, reusable definition of an external system (an MQTT broker, a database): host, port, options — plus an optionally attached credential. Connector nodes reference a connection instead of embedding addresses, so many nodes share one definition and a rotated credential is picked up on the next reconnect without redeploying. The **Test connection** button performs a real connect attempt (MQTT, all SQL dialects, MongoDB, Redis, Kafka, S3, Modbus TCP, OPC-UA) and shows the real error text if it fails.

Environment profiles. A flow can declare variables (for example a broker hostname); a profile is a named value set (dev/test/prod). Pick a profile in the editor's deploy row — its values resolve into the flow at deploy time (available in expressions as `env . NAME`), and a missing value blocks the deploy with a clear error.

4. The flow editor

The editor is where you build. Left: the searchable, categorized **palette** (drag a type onto the canvas; star favorites; recently used types surface automatically). Center: the infinite **canvas** — pan by dragging empty space, zoom with the wheel, toggle grid snap and minimap. Top: deploy target, per-flow log level, environment profile, dead letters, undo/redo, disable flow, debug mode, tutorial, save, export, and deploy.

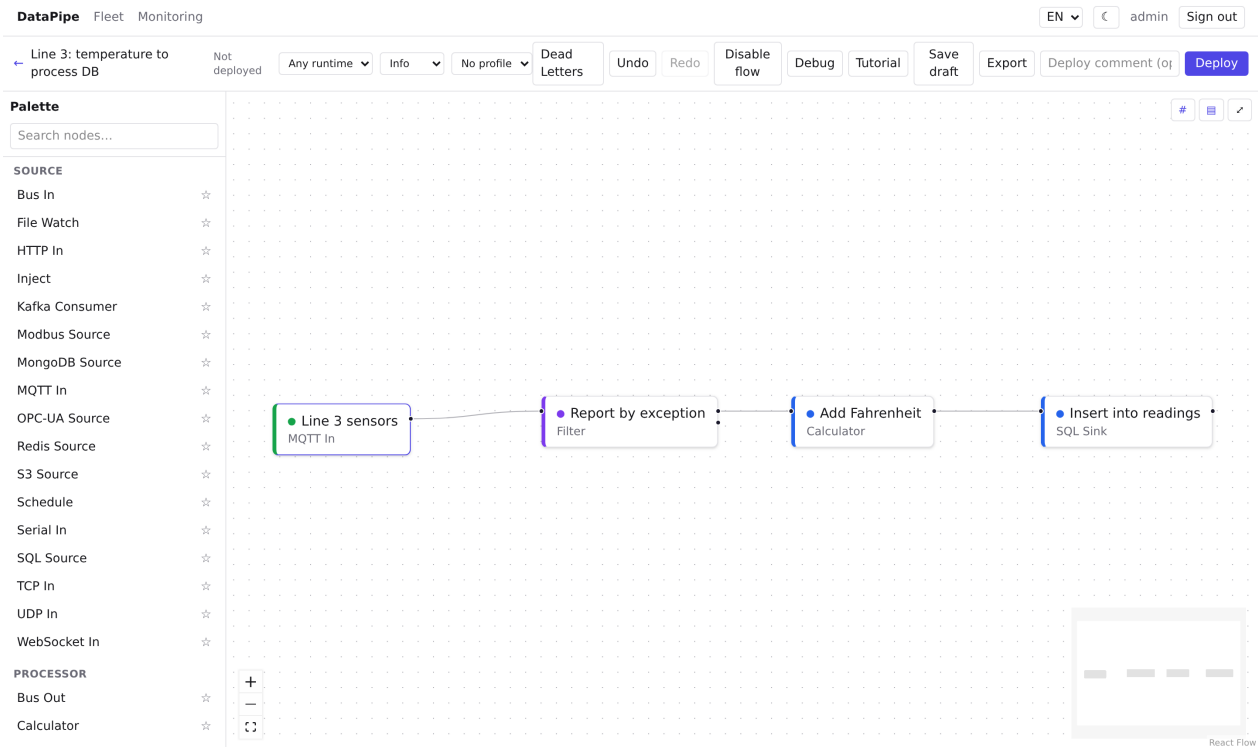


Figure 4 — The flow editor: palette (left), canvas with a wired flow, toolbar (top).

Wiring. Drag from a node's output port to another node's input port; valid targets highlight. Wires can be detached and reattached; deleting a node heals its wires. Nodes show their category color (sources green, processors blue, sinks orange), a status dot, and the user-assigned name.

Configuring nodes. Selecting a node opens the right-hand configuration panel — generated automatically from the node type's schema, so every node presents consistent controls with inline validation. The

Description tab holds your notes; the **Inspect** tab is the live-data view of section 6.

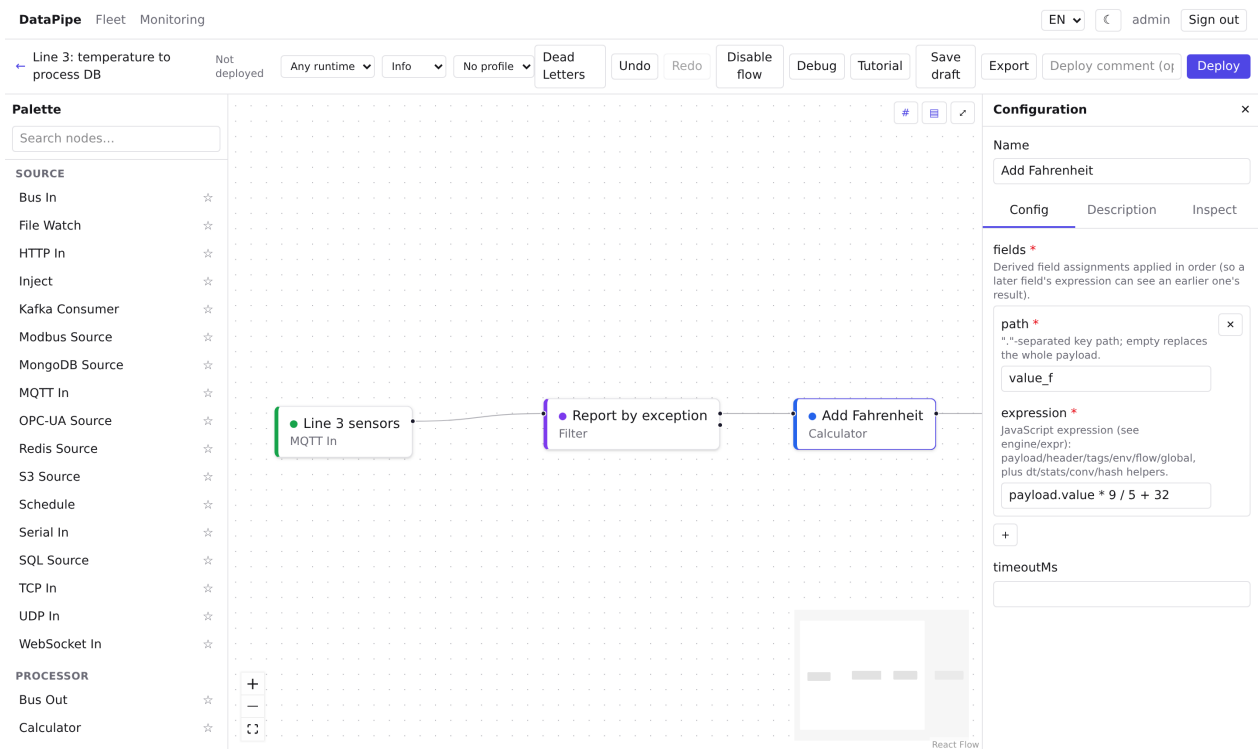


Figure 5 — Schema-generated configuration form with expression fields (Calculator node).

Editing commands.

Action	macOS	Windows/Linux
Undo / Redo (100 steps)	Cmd+Z / Shift+Cmd+Z	Ctrl+Z / Shift+Ctrl+Z
Copy / Paste (also across flows)	Cmd+C / Cmd+V	Ctrl+C / Ctrl+V
Duplicate selection	Cmd+D	Ctrl+D
Delete selection (wires heal)	Delete / Backspace	Delete / Backspace

Expressions. Any string parameter can switch from literal to expression. Expressions are sandboxed JavaScript over payload, header, tags, env, flow and global context, plus date/time, statistics, conversion, and hashing helpers (see docs/Expression-Language.md).

Saving and deploying. **Save draft** stores your work; **Deploy** validates the flow (broken wires, missing configuration, and mode violations block with a navigable error list), snapshots an immutable version, and pushes it to the runtime. The runtime hot-swaps only changed nodes — untouched nodes keep running. Every deploy creates a version; the REST API offers history and one-click rollback. Whole flows and single nodes can be disabled without deleting them.

5. Node reference

The palette contents at the state this manual covers. Every node additionally has the uniform error policy (fail, retry with backoff, route to error port, discard, or store-and-forward on edge devices) and a per-input overflow policy for backpressure.

Sources (streaming)

Node	Purpose
Inject	Emits a configurable test datagram, once or on an interval — ideal while building
Schedule	Time trigger: fixed interval or 6-field cron expression
MQTT In	Subscribes to broker topics (wildcards, QoS 0-2)

Node	Purpose
File Watch	Watches directories; parses CSV/TSV, JSON, JSON Lines, XML, Excel; post-actions (move/rename/delete/marker)
SQL Source	PostgreSQL/MySQL/MSSQL/SQLite: one-shot, periodic, or incremental with watermark column
MongoDB / Redis Source	find/aggregate queries; key polling, pub/sub, streams
Kafka Consumer	Consumer group with offset management
S3 Source	Lists and parses new objects under a prefix (S3-compatible)
Modbus Source	TCP/RTU master: polling groups over coils/registers, typed decoding
OPC-UA Source	Subscription (monitored items) or polled reads
TCP/UDP/Serial In	Raw byte streams with framing (delimiter, fixed length, length prefix, timeout)
WebSocket In	Server (accept) or client (connect) mode
Bus In	Subscribes to named internal bus topics (wildcards, tag filters) — flow-to-flow input

Triggers (start tracked executions)

Node	Purpose
HTTP In	Exposes a webhook endpoint; each request starts one tracked execution; pair with HTTP Response
Error Trigger	Entry point of an error-handler flow; receives unhandled errors of other flows

Processors

Node	Purpose
Set	Declarative field operations without code
Script	Sandboxed JavaScript with datagram and state access; multiple output ports; console output to debug sidebar
Calculator	Derived fields from expressions (unit conversion, math)
Convert / Template	JSON/XML/CSV/binary/base64 conversions; text rendering from placeholders
Window/Aggregate	Tumbling, sliding, session windows; sum/mean/min/max/count/stddev/... grouped by key
Switch / Filter	Rule-based routing to dynamic ports; pass/drop by predicate or deadband (report by exception)
Merge/Join / Split/Batch / Loop	Combine branches; split arrays or collect batches; iterate with a loop-back wire
Delay/Throttle	Fixed delay or rate limiting
Lookup / State	Enrich from SQL/HTTP/static tables with caching; named node/flow/global state
Try/Catch, Stop-and-Error	Visual error scope with catch port; deliberately fail an execution
HTTP Request	Full REST client usable mid-flow (auth, retry, pagination)

Sinks

Node	Purpose
Debug Log	Pushes selected values to the debug sidebar (console output optional)
MQTT Out / Kafka Out	Publish with QoS/retain; produce with key and headers — topics from expressions
HTTP Response / WebSocket Out	Reply to the exact triggering HTTP In request; push to WS peers

Node	Purpose
SQL Sink	insert/update/upsert/delete/exec; transactional batches; RETURNING merged back
MongoDB / Redis / S3 Sink	Document writes; key set/publish/stream-add; object upload
Modbus / OPC-UA Sink	Write registers and tags/values
TCP/UDP/Serial Out	Raw byte output with the same framing options
Bus Out	Publish to named internal bus topics — hand data to other flows

6. Watching your data live

DataPipe's signature feature: click on any node and see the real data flowing through it, without redeploying and without external tools. Live inspection requires the Operator role or higher. Enable **Debug** mode in the toolbar to activate wire animation, counters, and the sidebar; the runtime only captures and forwards data for flows somebody is actually watching, and the stream is rate-limited so debugging never overloads production.

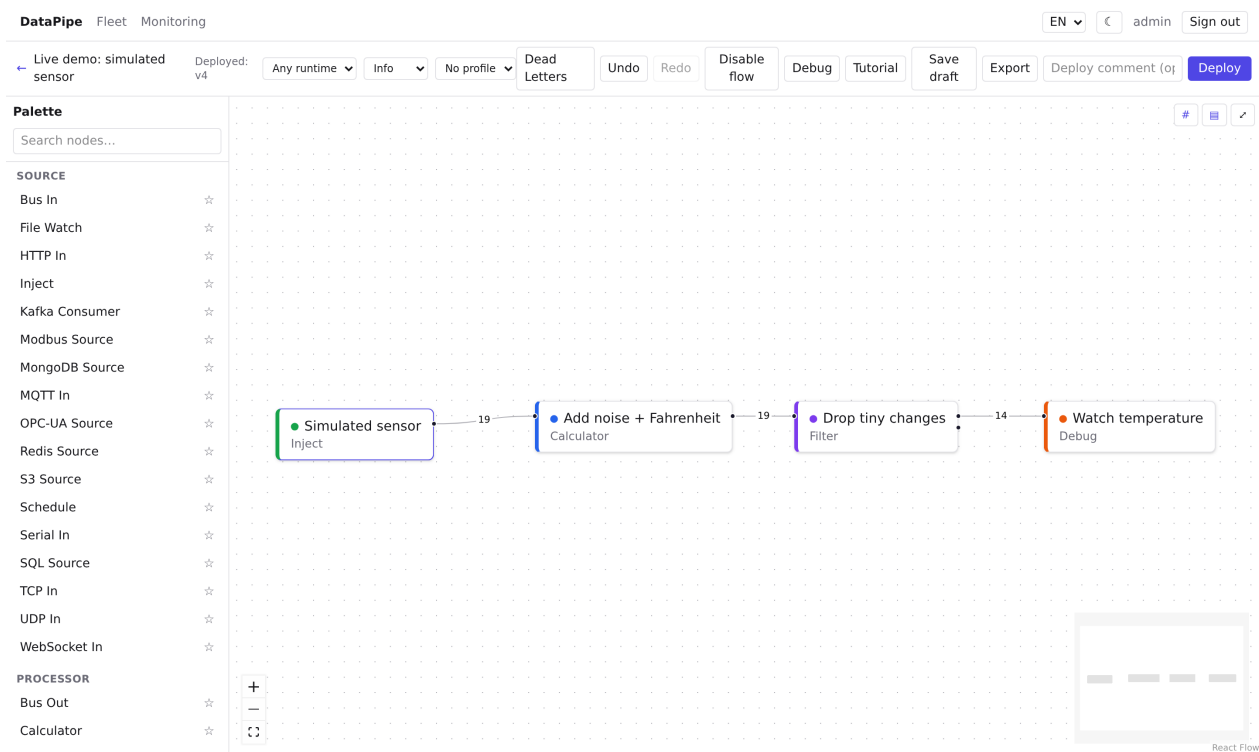


Figure 6 — A deployed flow in debug mode: live delivered-counters on every wire.

The Inspector. Select a node and open its **Inspect** tab. **Live data** shows the most recent datagrams through the node's inputs and outputs (a ring buffer of the last 50, kept runtime-side — inspection works immediately, no redeploy), as an expandable tree or raw JSON, with click-to-copy for any path or value. Large payloads arrive truncated with a load-full option.

DataPipe Fleet Monitoring EN admin Sign out

Live demo: simulated sensor Deployed: v4 Any runtime Info No profile Dead Letters Undo Redo Disable flow Debug Tutorial Save draft Export Deploy comment (or) Deploy

Palette

Search nodes...

SOURCE

- Bus In
- File Watch
- HTTP In
- Inject
- Kafka Consumer
- Modbus Source
- MongoDB Source
- MQTT In
- OPC-UA Source
- Redis Source
- S3 Source
- Schedule
- Serial In
- SQL Source
- TCP In
- UDP In
- WebSocket In

PROCESSOR

- Bus Out
- Calculator

Configuration

Name: Add noise + Fahrenheit

Run once

Input payload (JSON)

`{}`

Live data

out · out · GOOD	7:45:37 PM
sensor: TT-101 value: 20.83 value_f: 69.49	
in · in · GOOD	7:45:37 PM
sensor: TT-101 value: 21.5	
out · out · GOOD	7:45:36 PM
sensor: TT-101 value: 20.98 value_f: 69.76	
in · in · GOOD	7:45:36 PM
sensor: TT-101 value: 21.5	
out · out · GOOD	7:45:35 PM
sensor: TT-101 value: 22.21 value_f: 71.98	

Figure 7 — Inspector with live datagrams: port, quality, payload, timestamps.

Run once and pinning. The Inspect tab can also execute a single processor node against a JSON payload you type in — before the flow is even deployed — and **pin** captured outputs for reference while configuring. Source nodes offer **Fetch sample now** instead: run the source in isolation (up to 10 records / 10 seconds) to check a topic or query with real data before wiring the rest.

The debug sidebar. Debug Log nodes push selected values to a bottom-docked sidebar with per-node filtering, pause, and clear — the counterpart to Node-RED's debug panel.

DataPipe Fleet Monitoring EN admin Sign out

Live demo: simulated sensor Deployed: v4 Any runtime Info No profile Dead Letters Undo Redo Disable flow Debug Tutorial Save draft Export Deploy comment (or) Deploy

Palette

Search nodes...

SOURCE

- Bus In
- File Watch
- HTTP In
- Inject
- Kafka Consumer
- Modbus Source
- MongoDB Source
- MQTT In
- OPC-UA Source
- Redis Source
- S3 Source
- Schedule
- Serial In
- SQL Source
- TCP In
- UDP In
- WebSocket In

PROCESSOR

- Bus Out
- Calculator

Configuration

Name: Add noise + Fahrenheit

Run once

Input payload (JSON)

`{}`

Live data

out · out · GOOD	7:45:40 PM
sensor: TT-101 value: 20.33 value_f: 68.59	
in · in · GOOD	7:45:40 PM
sensor: TT-101 value: 21.5	
out · out · GOOD	7:45:39 PM
sensor: TT-101 value: 20.4 value_f: 68.72	
in · in · GOOD	7:45:39 PM
sensor: TT-101 value: 21.5	
out · out · GOOD	7:45:38 PM
sensor: TT-101 value: 20.44 value_f: 68.79	

Debug

All nodes Pause Clear x

temperature	7:45:38 PM
{"sensor": "TT-101", "value": 20.44, "value_f": 68.79}	
temperature	7:45:36 PM
{"sensor": "TT-101", "value": 20.98, "value_f": 69.76}	
temperature	7:45:35 PM
{"sensor": "TT-101", "value": 22.21, "value_f": 71.98}	

Figure 8 — Debug sidebar (bottom) with live entries, plus the Inspector on the right.

7. Triggered workflows and error handling

Flows starting with a trigger node create one tracked **execution** per event. The **Executions** page (linked from the flow editor) lists them with status, duration, and trigger; filter by status or jump to the flow's **dead letters**.

DataPipeFleetMonitoring

ENadminSign out

← Executions

All statusesDead letters

01KX3ZW60AXZAFRJY566R6SH19	0 ms	Failed
Trigger: webhook		
01KX3ZW606GVS3W9PM4NYXBZT3	0 ms	Success
Trigger: webhook		
01KX3ZW602JKB3NRAMR8S7H1NS	0 ms	Success
Trigger: webhook		
01KX3ZW5ZY99AFC5JRXBQG6S94	0 ms	Success
Trigger: webhook		
01KX3ZW5ZTEKCQ8RDPQ9H077V0	0 ms	Success
Trigger: webhook		
01KX3ZW5ZNBKKPE1GNF508ADR7	0 ms	Success
Trigger: webhook		
01KX3ZW5ZE5D4WZBKHG3ETWP3Q	0 ms	Success
Trigger: webhook		
01KX3ZT5CM5B4S9VQRCQ5XRXF8	0 ms	Failed
Trigger: webhook		
01KX3ZT2EH1Z1KZ8A4VB7WZMBP	0 ms	Failed
Trigger: webhook		
01KX3ZS2GB57WQB0YT5MVRSTD	0 ms	Failed
Trigger: webhook		
01KX3ZSMJ48ZR6F4ZN4WXS1A9G	0 ms	Failed
Trigger: webhook		
01KX3ZSSKXMMQWH2867V00TM9T	0 ms	Failed
Trigger: webhook		
01KX3ZSPNS7KCX6FHD57S9GWDD	0 ms	Failed
Trigger: webhook		
01KX3ZSKQH40BSQB53X7RW63RF	0 ms	Failed
Trigger: webhook		
01KX3Z9BVZWBCQPWG8DPNN683	0 ms	Failed
Trigger: webhook		

Figure 9 — Execution history of a webhook flow: successes and failures.

Opening an execution shows the **node trace**: each node's input and output, and for failures the exact error with its node. **Re-run from start** repeats the whole execution with the same trigger data; **Re-run from this node** resumes at the failed node after you fix the cause.

DataPipeFleetMonitoring

ENadminSign out

← Execution 01KX3ZW60AXZAFRJY566R6SH19 Failed

Trigger: webhook0 msReason: script: Error: invalid quantity: 0 at script:3:43(32)

Re-run from start

Node trace

c2Re-run from this node

Error: script: Error: invalid quantity: 0 at script:3:43(32)

Input

▶ header:

▶ payload:

Outputs

null

Figure 10 — A failed execution: error reason, node trace, re-run buttons.

Error handling beyond executions. Every node has an error policy (retry with backoff, error output port, discard, fail). Unhandled errors go to a flow-level error handler — a flow starting with **Error Trigger** — and undeliverable or expired datagrams land in the per-flow **dead letter** store, browsable and re-injectable after the underlying problem is fixed. A crashed runtime restores all deployed flows automatically on restart.

8. Monitoring and fleet

The **Monitoring** page is the built-in operations view: runtime health (online state, CPU, memory, flow count), per-flow health with recent failures, a connection status board with test buttons, and **alert rules** (connection down, edge runtime offline) that fire to webhooks. Prometheus-compatible metrics are exposed for plant-level monitoring systems.

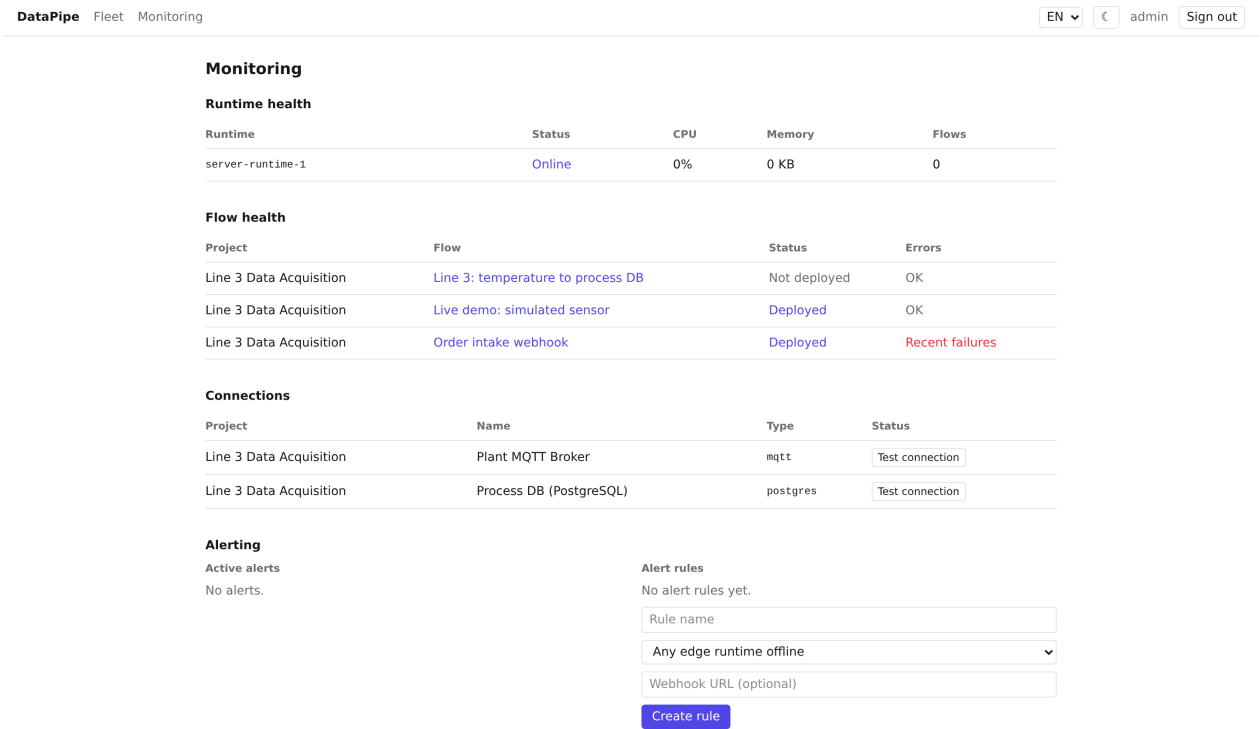


Figure 11 — Monitoring: runtime health, flow health, connections, alerting.

The **Fleet** page manages runtimes: the central server runtime and any number of **edge runtimes** installed close to the machines (a single small binary, also for ARM64 Linux). Edge devices enroll with one-time tokens, report health, and are organized into **runtime groups**; a flow's deploy target (editor toolbar) can be a specific group, so "deploy to all line-3 gateways" is one click. Edge runtimes keep running autonomously when the network is down and buffer outbound data durably (store-and-forward) until reconnect.

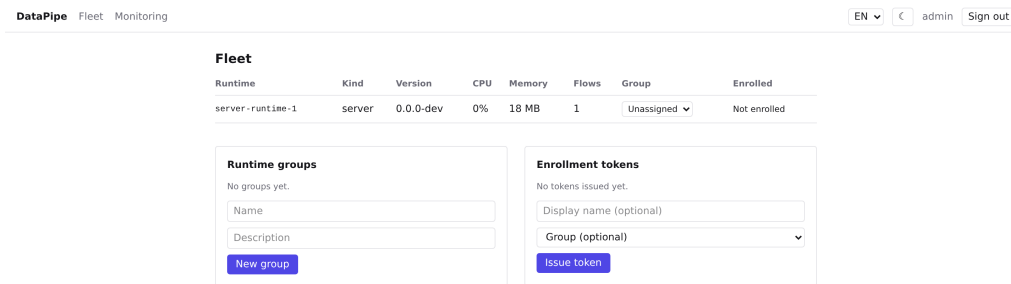


Figure 12 — Fleet: runtimes with health, runtime groups, enrollment tokens.

9. A worked example, end to end

The flow from Figures 4 and 5, built in about five minutes:

1. On the project page, create two connections: your MQTT broker and your target database; press **Test connection** on both.
2. Create a flow; drag **MQTT In** onto the canvas, pick the broker connection, set the topic filter (plant/line3/+/temperature); use **Fetch sample now** to see real readings.
3. Add a **Filter** in deadband mode (report by exception) so only meaningful changes pass.
4. Add a **Calculator** for derived fields — here payload.value * 9 / 5 + 32.
5. Add an **SQL Sink**, pick the database connection, choose insert mode, map the columns.
6. Wire the four nodes, **Save draft**, **Deploy** — then enable **Debug** and watch datagrams flow.

10. Tips and troubleshooting

Symptom	What to check
Deploy rejected (validation)	The error list names every problem: broken wires, missing required config, missing environment variable values, mixed trigger/source entry nodes
Deploy: "no runtime connected"	The runtime process isn't registered — ask your administrator, or check the Fleet page
Node shows error status	Hover the status dot for the message; open Inspect for the failing datagrams; check the node's error policy
No live data in the Inspector	Is the flow deployed and producing? Are you at least Operator in the project? Debug data is sampled at high rates — counters stay exact
Connection test fails	The real dial error is shown; verify host/port and reachability from the server (the test runs on the control plane)
Webhook returns nothing	Pair every HTTP In with an HTTP Response node; failures respond after the configured response timeout — check the execution's node trace
Where did a datagram go?	Filter drop ports, dead letters (per-flow page), and per-wire drop counters in debug mode account for every datagram — nothing is silently lost

Version note. DataPipe is under active development. This manual describes the state after development increment 10; SECS/GEM connectivity (semiconductor equipment, HSMS/GEM host) is the next milestone. The authoritative feature state is always docs/User-Guide.md in the repository.